-- QUESTION 1 — Variables & DBMS_OUTPUT

-- Student: SANO DANY ID: 29271

Task 1.1

SET SERVEROUTPUT ON; DECLARE v_course_name VARCHAR2(100) :=

'Database Management Systems'; v_credits NUMBER(1) := 4;

v_start_date DATE := DATE '2024-09-01'; v_heavy_course BOOLEAN;

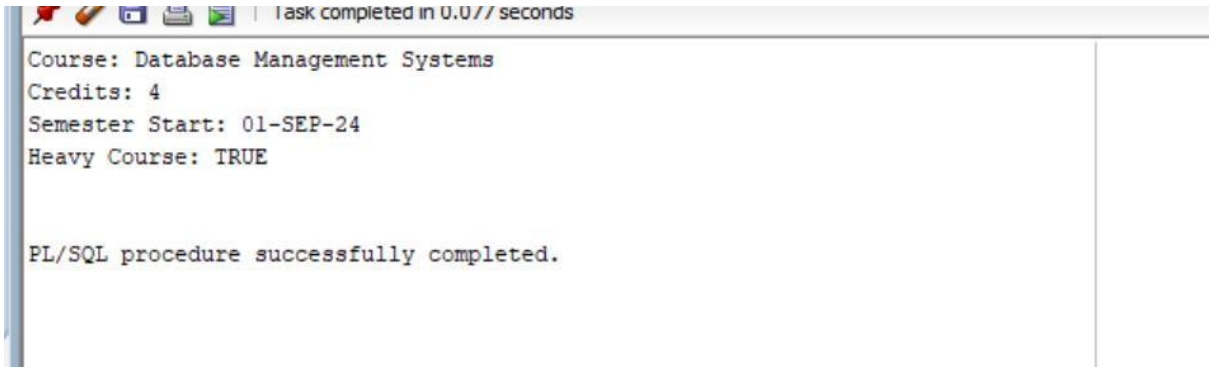
BEGIN

-- Boolean evaluation v_heavy_course

:= (v_credits >= 3);

```
DBMS_OUTPUT.PUT_LINE('Course: ' || v_course_name);
DBMS_OUTPUT.PUT_LINE('Credits: ' || v_credits);
DBMS_OUTPUT.PUT_LINE('Semester Start: ' ||
                TO_CHAR(v_start_date,'DD-MON-YY'));

IF v_heavy_course THEN
   DBMS_OUTPUT.PUT_LINE('Heavy Course: TRUE');
ELSE
   DBMS_OUTPUT.PUT_LINE('Heavy Course: FALSE');
END IF; END;
```

/



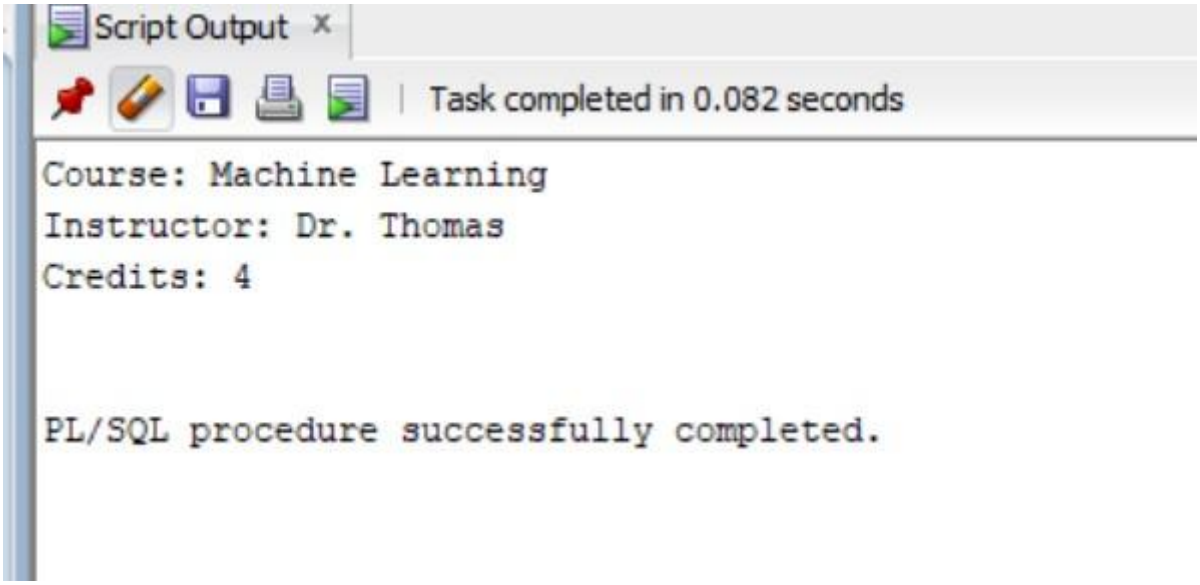
Task 1.2

SET SERVEROUTPUT ON; DECLARE

v_course_name courses.course_name%TYPE;

v_instructor courses.instructor%TYPE;

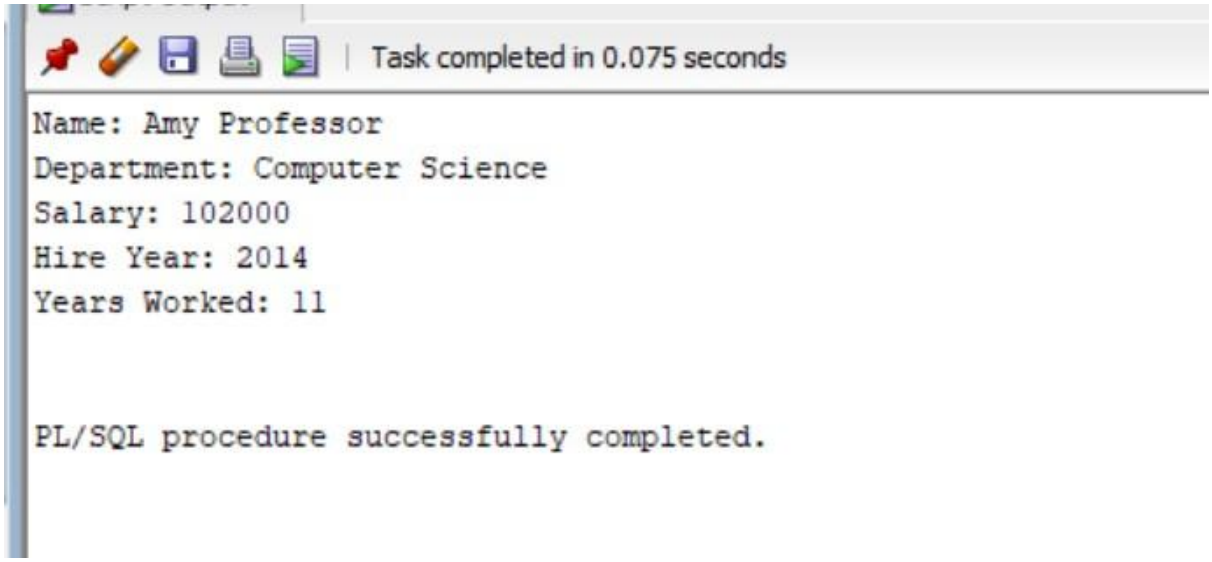
```
v_credits courses.credits%TYPE;

v_max_students courses.max_students%TYPE;

BEGIN

SELECT course_name, instructor, credits, max_students

INTO v_course_name, v_instructor, v_credits, v_max_students

FROM courses

WHERE course_id = 110;

DBMS_OUTPUT.PUT_LINE('Course: ' || v_course_name);
DBMS_OUTPUT.PUT_LINE('Instructor: ' || v_instructor);
DBMS_OUTPUT.PUT_LINE('Credits: ' || v_credits);

IF v_max_students < 20 THEN
   DBMS_OUTPUT.PUT_LINE('Limited seats available!');
END IF; END;

/
```



Task 1.3

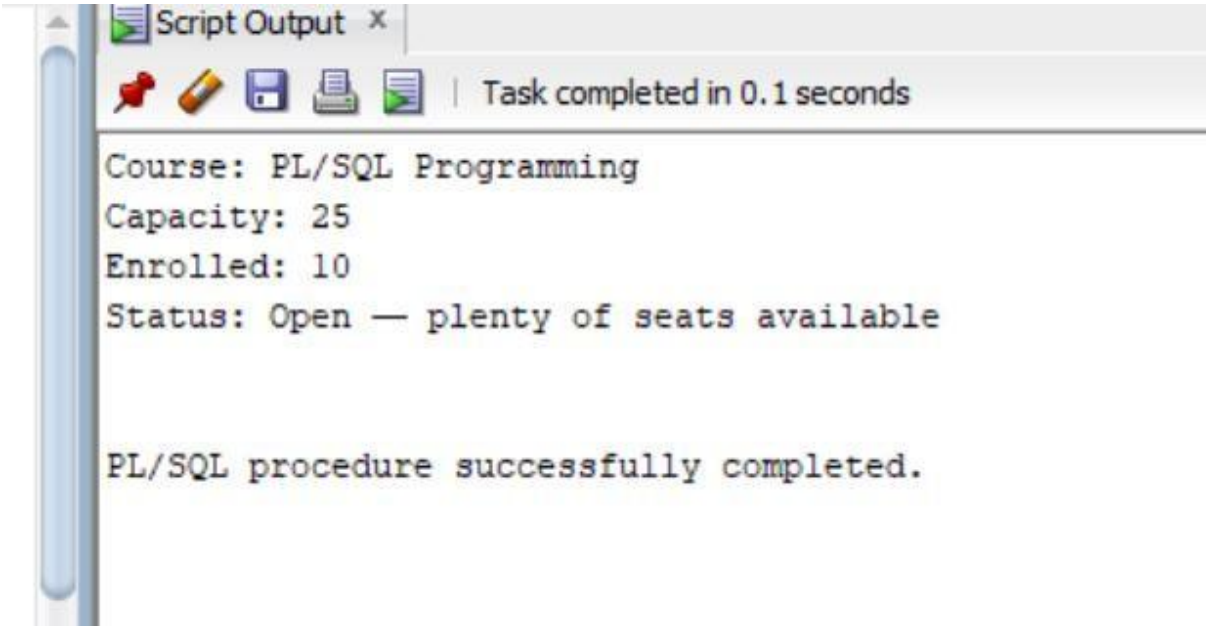
```
set serveroutput on; DECLARE

v_emp employees%ROWTYPE;

v_years NUMBER;

BEGIN

SELECT *
```

```
INTO v_emp

FROM employees WHERE

employee_id = 22;

v_years := FLOOR(
   MONTHS_BETWEEN(SYSDATE, v_emp.hire_date)/12
);

DBMS_OUTPUT.PUT_LINE('Name: ' ||
   v_emp.first_name || ' ' || v_emp.last_name);

DBMS_OUTPUT.PUT_LINE('Department: ' || v_emp.department);
DBMS_OUTPUT.PUT_LINE('Salary: ' || v_emp.salary);
DBMS_OUTPUT.PUT_LINE('Hire Year: ' ||
   EXTRACT(YEAR FROM v_emp.hire_date));

DBMS_OUTPUT.PUT_LINE('Years Worked: ' || v_years); END;

/
```



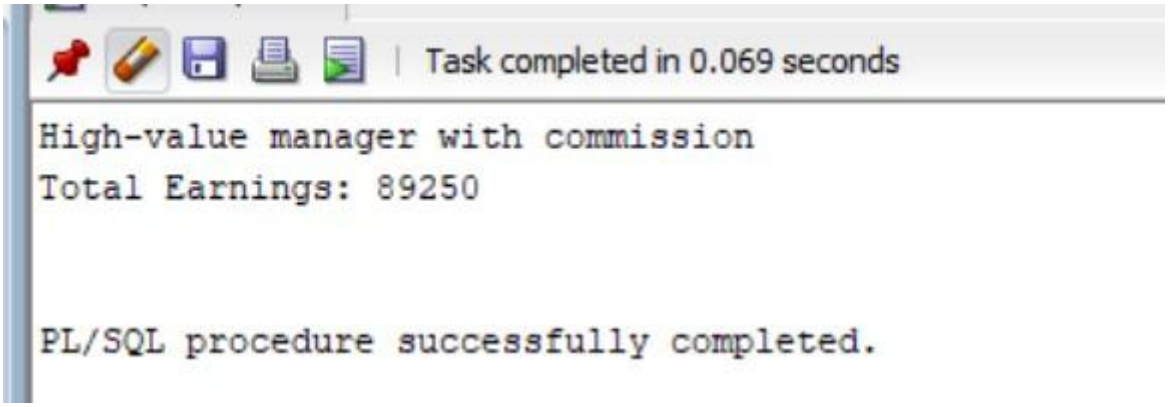

```
-- ==========================================

-- Question 2 — IF / ELSIF / ELSE Conditions

-- Student: SANO DANY Levy ID: 29271

-- ==========================================

-- PART 2.1  set

serveroutput on;
```

```
DECLARE  v_name courses.course_name%TYPE;  v_max

courses.max_students%TYPE; v_count NUMBER;

v_status VARCHAR2(50);

BEGIN

SELECT course_name, max_students INTO v_name, v_max FROM courses WHERE course_id = 103;

SELECT COUNT(*)
INTO v_count
FROM enrollments
WHERE course_id = 103;

IF v_count < v_max*0.5 THEN
   v_status := 'Open — plenty of seats available';

ELSIF v_count BETWEEN v_max*0.5 AND v_max*0.8 THEN
   v_status := 'Filling up — limited seats';

ELSIF v_count > v_max*0.8
     AND v_count < v_max THEN
v_status := 'Almost full — enroll soon';

ELSE
   v_status := 'FULL — enrollment closed';
END IF;

DBMS_OUTPUT.PUT_LINE('Course: '||v_name);
DBMS_OUTPUT.PUT_LINE('Capacity: '||v_max);
DBMS_OUTPUT.PUT_LINE('Enrolled: '||v_count);
DBMS_OUTPUT.PUT_LINE('Status: '||v_status);

END;

 /
```



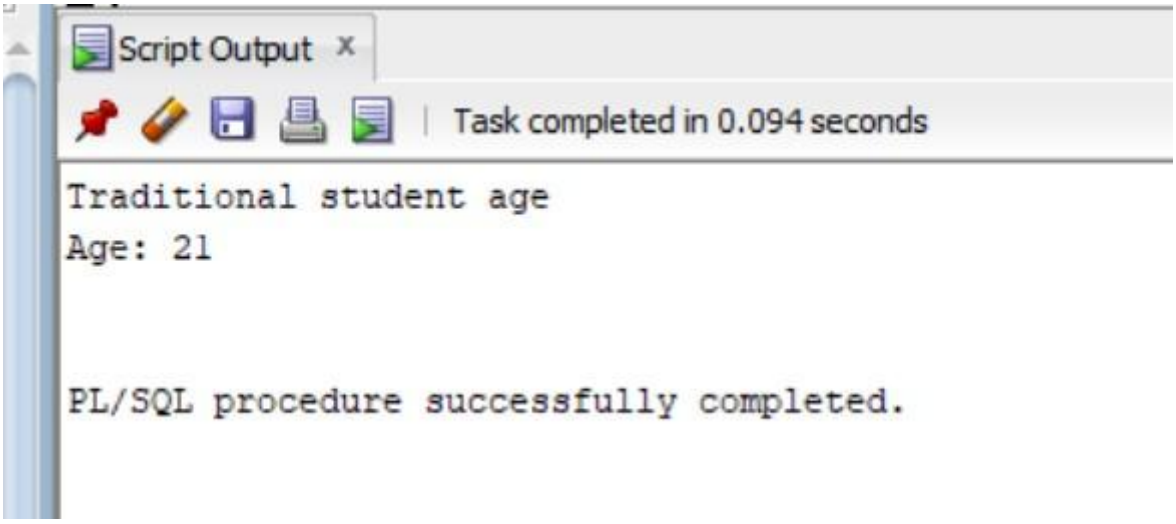

```
-- Task 2.2 set serveroutput

on; DECLARE v_emp

employees%ROWTYPE;

v_total NUMBER;

BEGIN

SELECT * INTO v_emp

FROM employees

WHERE employee_id = 1;

IF v_emp.commission_pct IS NOT NULL
  AND v_emp.salary > 80000 THEN

   DBMS_OUTPUT.PUT_LINE(
      'High-value manager with commission');

ELSIF v_emp.commission_pct IS NOT NULL
  AND v_emp.salary <= 80000 THEN

   DBMS_OUTPUT.PUT_LINE(
      'Manager with commission — standard tier');

ELSIF v_emp.commission_pct IS NULL
  AND v_emp.salary > 90000 THEN

   DBMS_OUTPUT.PUT_LINE(
      'Senior employee — consider adding commission incentive');
```

```
ELSE
   DBMS_OUTPUT.PUT_LINE('Standard employee profile');
END IF;

v_total :=
v_emp.salary +
   (v_emp.salary * NVL(v_emp.commission_pct,0));

DBMS_OUTPUT.PUT_LINE('Total Earnings: ' || v_total); END;

/
```



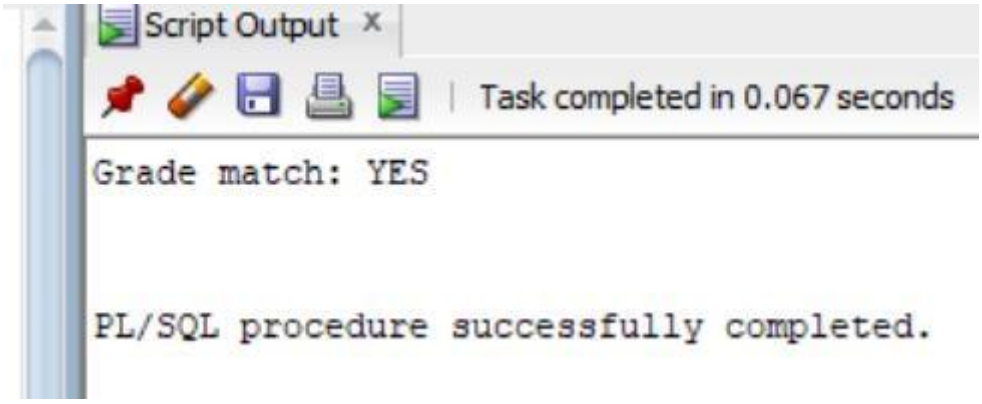

```
/

-- task 2.3 set serveroutput on;

DECLARE v_student

students%ROWTYPE; v_age

NUMBER;

BEGIN

SELECT * INTO v_student

FROM students

WHERE student_id = 1020;

v_age :=
  FLOOR(MONTHS_BETWEEN(SYSDATE,
      v_student.date_of_birth)/12);

IF v_age < 18 THEN
   DBMS_OUTPUT.PUT_LINE(
      'Minor student — parental consent required');

ELSIF v_age BETWEEN 18 AND 25 THEN
```

```
   DBMS_OUTPUT.PUT_LINE(
      'Traditional student age');

ELSE
   DBMS_OUTPUT.PUT_LINE(
      'Mature student — eligible for evening program');
END IF;

IF EXTRACT(YEAR FROM v_student.enrollment_date)=2023 THEN
   DBMS_OUTPUT.PUT_LINE(
      'First-year cohort (2023)');
END IF;

DBMS_OUTPUT.PUT_LINE('Age: '||v_age); END;

/
```



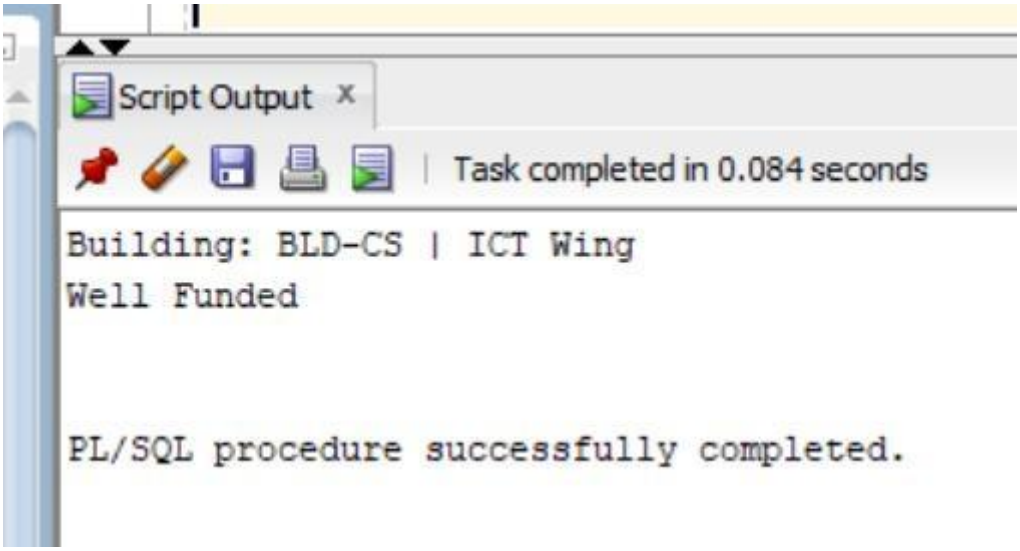

```
-- ==========================================

-- Question 3 — CASE Statement

-- Student: SANO DANY ID: 29271

-- ==========================================

-- Task 3.1 set

serveroutput on;

DECLARE

v_score enrollments.score%TYPE;

v_stored enrollments.grade%TYPE;

v_grade VARCHAR2(20); BEGIN
```

```
SELECT score, grade

INTO v_score, v_stored

FROM enrollments WHERE

enrollment_id = 5;

v_grade :=
   CASE
      WHEN v_score IS NULL THEN 'Not yet graded'
      WHEN v_score >= 90 THEN 'A'
WHEN v_score >= 80 THEN 'B'
      WHEN v_score >= 70 THEN 'C'
      WHEN v_score >= 60 THEN 'D'
      ELSE 'F'
   END;

IF v_grade = v_stored THEN
   DBMS_OUTPUT.PUT_LINE('Grade match: YES');
ELSE
   DBMS_OUTPUT.PUT_LINE(
      'Grade match: NO — stored '
      || v_stored || ', computed ' || v_grade);
END IF; END;

/
```




```
-- task 3.2 set

serveroutput on;

DECLARE v_dep

departments%ROW

TYPE; v_building
```

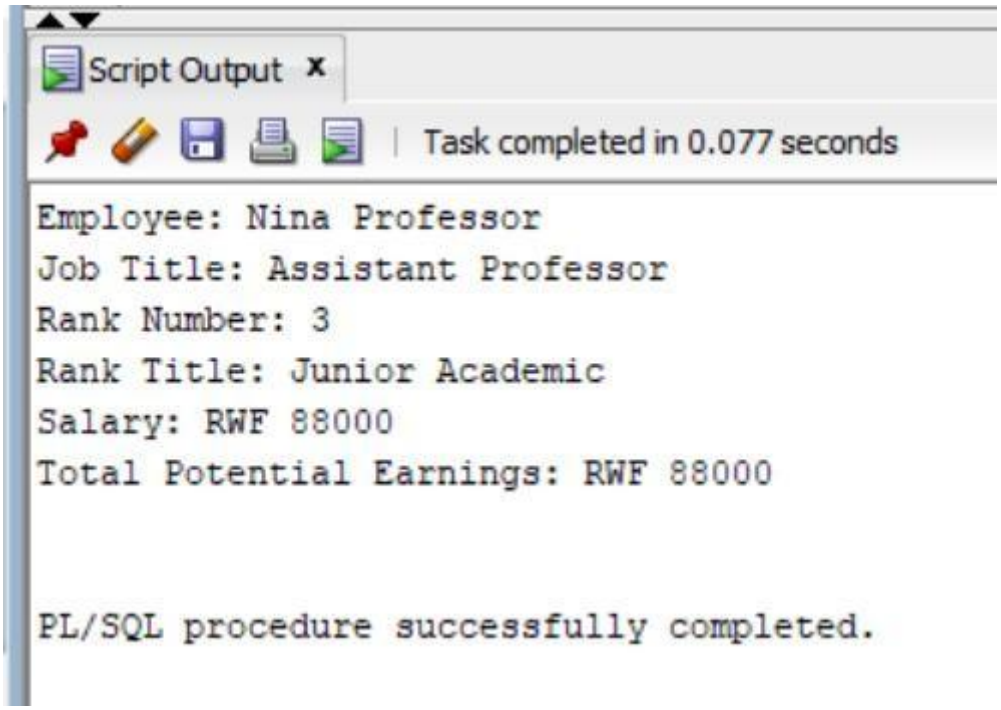
```
VARCHAR2(50);

BEGIN

SELECT * INTO v_dep

FROM departments

WHERE department_name='Computer Science';

v_building :=
   CASE v_dep.department_name
      WHEN 'Computer Science' THEN 'BLD-CS | ICT Wing'
      WHEN 'Mathematics' THEN 'BLD-MT | Science Wing'
WHEN 'Business' THEN 'BLD-BS | Commerce Wing'
      WHEN 'Engineering' THEN 'BLD-EN | Technical Wing'
      WHEN 'Psychology' THEN 'BLD-PS | Humanities Wing'
      ELSE 'BLD-GN | General Wing'
   END;

DBMS_OUTPUT.PUT_LINE('Building: '||v_building);

DBMS_OUTPUT.PUT_LINE(
   CASE
      WHEN v_dep.budget > 600000 THEN 'Well Funded'
      WHEN v_dep.budget BETWEEN 400000 AND 600000
         THEN 'Adequately Funded'
      ELSE 'Underfunded'
   END); END;

/
```




```
-- Task 3.3

SET SERVEROUTPUT ON;
```

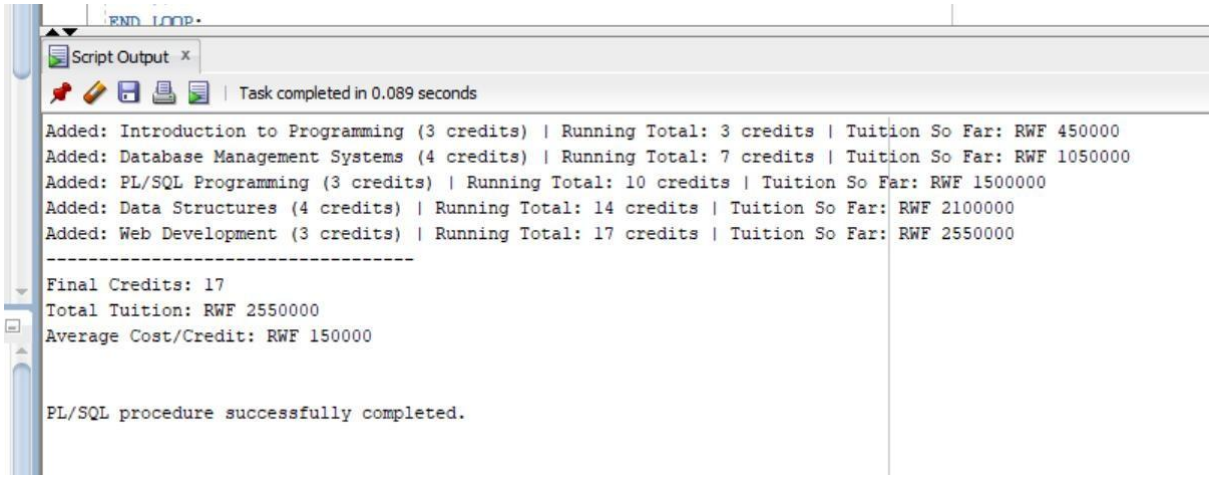
```
DECLARE

-- Rowtype variable to hold the entire employee row v_emp

employees%ROWTYPE;

-- Supporting variables
v_rank_number NUMBER; v_rank_title
VARCHAR2(50);
v_total_earnings employees.salary%TYPE; BEGIN

-- Fetch the full row for employee_id = 14

SELECT *

INTO v_emp

FROM employees

WHERE employee_id = 14;

-- Assign rank number and title using CASE
CASE
   WHEN v_emp.job_title = 'Full Professor' THEN
      v_rank_number := 1;
      v_rank_title := 'Senior Academic';
   WHEN v_emp.job_title = 'Associate Professor' THEN
      v_rank_number := 2;
v_rank_title := 'Mid Academic';
   WHEN v_emp.job_title = 'Assistant Professor' THEN
      v_rank_number := 3;
      v_rank_title := 'Junior Academic';
   WHEN v_emp.job_title LIKE '%Developer%' OR v_emp.job_title LIKE '%Engineer%' THEN
v_rank_number := 4;
      v_rank_title := 'Technical Staff';
   WHEN v_emp.job_title LIKE '%Admin%' OR v_emp.job_title LIKE '%Coordinator%' THEN
v_rank_number := 5;
      v_rank_title := 'Administrative Staff';
ELSE
      v_rank_number := 6;
v_rank_title := 'General Staff';
END CASE;

-- Print employee details
DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp.first_name || ' ' || v_emp.last_name);
DBMS_OUTPUT.PUT_LINE('Job Title: ' || v_emp.job_title);
DBMS_OUTPUT.PUT_LINE('Rank Number: ' || v_rank_number);
DBMS_OUTPUT.PUT_LINE('Rank Title: ' || v_rank_title);
DBMS_OUTPUT.PUT_LINE('Salary: RWF ' || v_emp.salary);
```

```
-- Promotion eligibility check
IF v_rank_number <= 2 AND v_emp.salary < 100000 THEN
   DBMS_OUTPUT.PUT_LINE('Promotion eligible');
END IF;

-- Calculate and print total potential earnings v_total_earnings := v_emp.salary +
(v_emp.salary * NVL(v_emp.commission_pct, 0));
DBMS_OUTPUT.PUT_LINE('Total Potential Earnings: RWF ' || v_total_earnings); END;

/
```




```
-- ==========================================

-- Question 4 — Loops

-- Student: SANO DANY ID: 29271

-- ==========================================

-- task 4.1

SET SERVEROUTPUT ON;

DECLARE v_total_credits

NUMBER := 0; v_total_tuition

NUMBER := 0;

v_fee_per_credit CONSTANT

NUMBER := 150000;
```

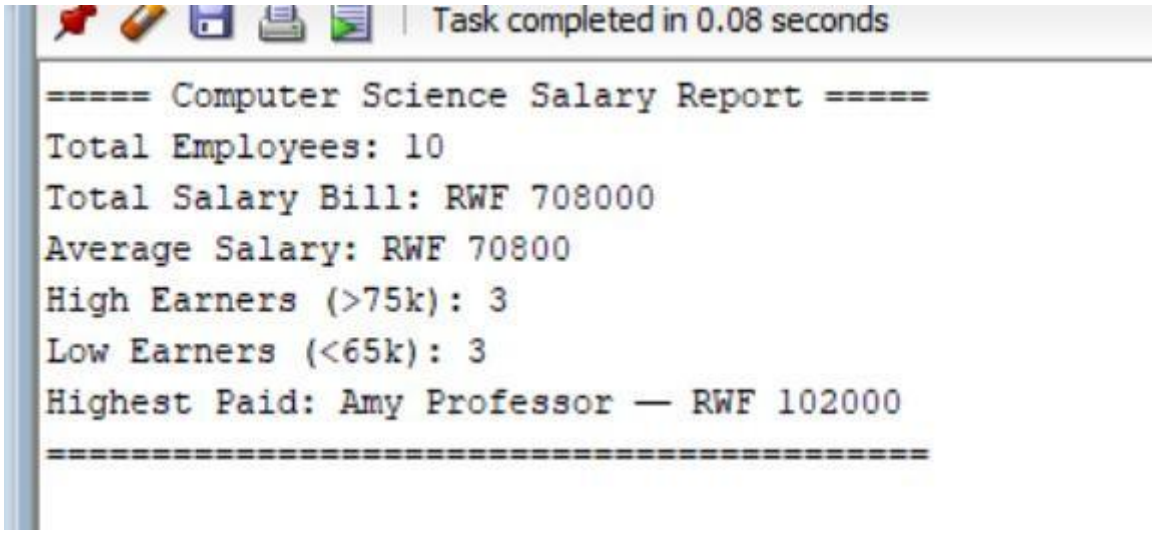
```
v_course_name courses.course_name%TYPE;
v_credits     courses.credits%TYPE;

CURSOR c_courses IS
   SELECT course_name, credits
   FROM courses
   ORDER BY course_id; BEGIN

OPEN c_courses;

LOOP
   FETCH c_courses INTO v_course_name, v_credits;
   EXIT WHEN c_courses%NOTFOUND OR v_total_credits >= 15;

   v_total_credits := v_total_credits + v_credits;
   v_total_tuition := v_total_tuition + (v_credits * v_fee_per_credit);

   DBMS_OUTPUT.PUT_LINE(
      'Added: ' || v_course_name ||
      ' (' || v_credits || ' credits) | Running Total: ' ||
v_total_credits || ' credits | Tuition So Far: RWF ' ||
v_total_tuition
   );
END LOOP;

CLOSE c_courses;

DBMS_OUTPUT.PUT_LINE('-----------------------------------');
DBMS_OUTPUT.PUT_LINE('Final Credits: ' || v_total_credits);
DBMS_OUTPUT.PUT_LINE('Total Tuition: RWF ' || v_total_tuition);
DBMS_OUTPUT.PUT_LINE('Average Cost/Credit: RWF ' ||
   ROUND(v_total_tuition / v_total_credits,2));
END;

/
```



-- task 4.2

```
SET SERVEROUTPUT ON;

DECLARE

-- Cursor to fetch all employees in Computer Science department

CURSOR cs_emp_cur IS

SELECT employee_id, first_name, last_name, salary

FROM employees

WHERE department = 'Computer Science';

-- Variables to hold cursor data v_emp_id
employees.employee_id%TYPE;       v_first
employees.first_name%TYPE;          v_last
employees.last_name%TYPE;
v_salary employees.salary%TYPE;

-- Totals and counters v_total_salary
NUMBER := 0; v_total_employees
NUMBER := 0; v_high_earners
NUMBER := 0;
v_low_earners NUMBER := 0;

-- Track highest paid employee
v_max_salary NUMBER := 0;
v_max_name VARCHAR2(100); BEGIN

-- Open cursor

OPEN cs_emp_cur;

-- Fetch first row
FETCH cs_emp_cur INTO v_emp_id, v_first, v_last, v_salary;

-- WHILE loop to process all rows
WHILE cs_emp_cur%FOUND LOOP
   -- Count employee
   v_total_employees := v_total_employees + 1;

   -- Accumulate salary
   v_total_salary := v_total_salary + v_salary;

   -- Check high earners
IF v_salary > 75000 THEN
      v_high_earners := v_high_earners + 1;
   END IF;
```

```
   -- Check low earners     IF
v_salary < 65000 THEN
      v_low_earners := v_low_earners + 1;
   END IF;

   -- Track highest paid
   IF v_salary > v_max_salary THEN
v_max_salary := v_salary;
      v_max_name := v_first || ' ' || v_last;
   END IF;

   -- Fetch next row
   FETCH cs_emp_cur INTO v_emp_id, v_first, v_last, v_salary;
END LOOP;

-- Close cursor
CLOSE cs_emp_cur;

-- Print report
DBMS_OUTPUT.PUT_LINE('===== Computer Science Salary Report =====');
DBMS_OUTPUT.PUT_LINE('Total Employees: ' || v_total_employees);
DBMS_OUTPUT.PUT_LINE('Total Salary Bill: RWF ' || v_total_salary);
DBMS_OUTPUT.PUT_LINE('Average Salary: RWF ' || ROUND(v_total_salary / v_total_employees, 2));
DBMS_OUTPUT.PUT_LINE('High Earners (>75k): ' || v_high_earners);
DBMS_OUTPUT.PUT_LINE('Low Earners (<65k): ' || v_low_earners);
DBMS_OUTPUT.PUT_LINE('Highest Paid: ' || v_max_name || ' — RWF ' || v_max_salary);
DBMS_OUTPUT.PUT_LINE('==========================================='); END;

/
```




```
-- Task 4.3

SET SERVEROUTPUT ON;

DECLARE
```

```
v_name VARCHAR2(100); v_gpa

students.gpa%TYPE;

v_enroll_count NUMBER;

v_perf_label VARCHAR2(40); v_load_label
VARCHAR2(30);

v_total_students NUMBER := 0;

v_summa NUMBER := 0;
v_magna NUMBER := 0;
v_cum   NUMBER := 0; v_sat
NUMBER := 0; v_prob
NUMBER := 0; BEGIN

FOR sid IN 1001..1010 LOOP

   SELECT first_name || ' ' || last_name, gpa
   INTO v_name, v_gpa
   FROM students
   WHERE student_id = sid;

   SELECT COUNT(*)
   INTO v_enroll_count
   FROM enrollments
   WHERE student_id = sid;

   -- GPA CASE
v_perf_label :=
      CASE
         WHEN v_gpa >= 3.8 THEN 'Summa Cum Laude'
         WHEN v_gpa >= 3.5 THEN 'Magna Cum Laude'
         WHEN v_gpa >= 3.2 THEN 'Cum Laude'
         WHEN v_gpa >= 2.5 THEN 'Satisfactory'
         ELSE 'Probation'
      END;

   -- counters
   CASE v_perf_label
      WHEN 'Summa Cum Laude' THEN v_summa := v_summa + 1;
      WHEN 'Magna Cum Laude' THEN v_magna := v_magna + 1;
      WHEN 'Cum Laude' THEN v_cum := v_cum + 1;
      WHEN 'Satisfactory' THEN v_sat := v_sat + 1;
      ELSE v_prob := v_prob + 1;
   END CASE;
```

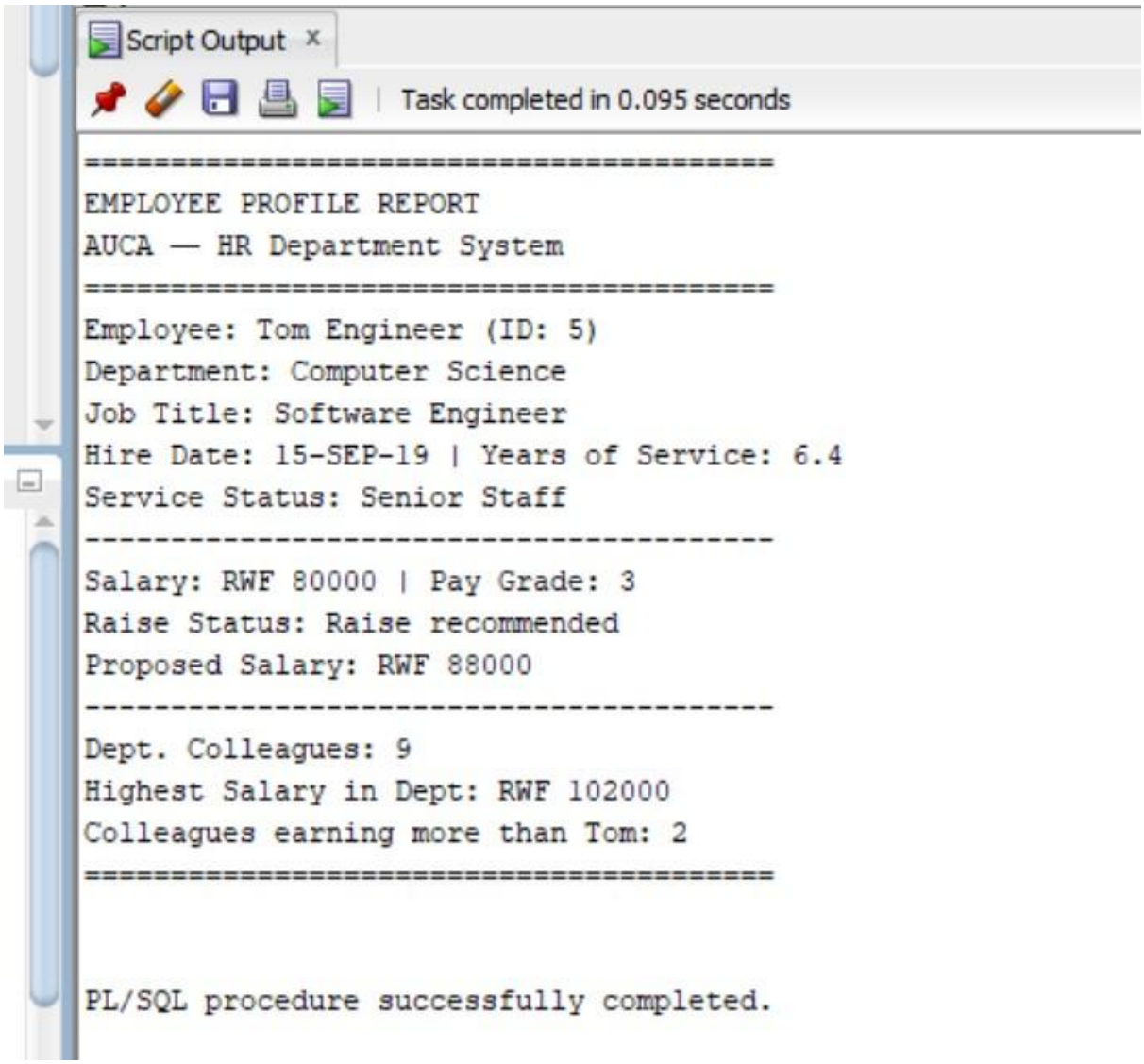
```
   -- enrollment load
v_load_label :=
      CASE
         WHEN v_enroll_count <= 1 THEN 'Under-enrolled'
         WHEN v_enroll_count <= 3 THEN 'Normal Load'
         ELSE 'Full Load'
      END;

   DBMS_OUTPUT.PUT_LINE(
      'ID:' || sid || ' | ' ||
      RPAD(v_name,18) ||
      ' | GPA:' || v_gpa ||
      ' | ' || RPAD(v_perf_label,18) ||
      ' | ' || v_load_label
   );

   v_total_students := v_total_students + 1;
END LOOP;

DBMS_OUTPUT.PUT_LINE('-----------------------------------');
DBMS_OUTPUT.PUT_LINE('Total Students: ' || v_total_students);
DBMS_OUTPUT.PUT_LINE('Summa: ' || v_summa);
DBMS_OUTPUT.PUT_LINE('Magna: ' || v_magna);
DBMS_OUTPUT.PUT_LINE('Cum Laude: ' || v_cum);
DBMS_OUTPUT.PUT_LINE('Satisfactory: ' || v_sat);
DBMS_OUTPUT.PUT_LINE('Probation: ' || v_prob); END;

/
```

Script Output

Task completed in 0.072 seconds

```
ID:1001 | Alice Johnson      | GPA:3.8 | Summa Cum Laude    | Full Load
ID:1002 | Bob Smith          | GPA:3.2 | Cum Laude          | Normal Load
ID:1003 | Carol White        | GPA:3.5 | Magna Cum Laude    | Normal Load
ID:1004 | David Green        | GPA:2.9 | Satisfactory       | Normal Load
ID:1005 | Emma Davis         | GPA:3.9 | Summa Cum Laude    | Normal Load
ID:1006 | Frank Miller       | GPA:3.4 | Cum Laude          | Normal Load
ID:1007 | Grace Wilson       | GPA:3.7 | Magna Cum Laude    | Normal Load
ID:1008 | Henry Moore        | GPA:3.1 | Satisfactory       | Normal Load
ID:1009 | Ivy Taylor         | GPA:3.6 | Magna Cum Laude    | Normal Load
ID:1010 | Jack Anderson      | GPA:2.8 | Satisfactory       | Normal Load
-----------------------------------
Total Students: 10
Summa: 2
Magna: 3
Cum Laude: 2
Satisfactory: 3
Probation: 0


PL/SQL procedure successfully completed.
```

```
-- ==========================================

-- Question 5 — Combined Challenge

-- Student: SANO DANY ID: 29271

-- ==========================================

SET SERVEROUTPUT ON;

DECLARE

-- Step 1: Employee Profile

v_emp     employees%ROWTYPE;

v_years_service           NUMBER;

v_service_status VARCHAR2(20);

-- Step 2: Salary Analysis v_pay_grade
NUMBER; v_raise_status
VARCHAR2(30);
v_proposed_salary employees.salary%TYPE;

-- Step 3: Department Summary
v_total_colleagues NUMBER := 0;
v_highest_salary NUMBER := 0;
v_higher_than_tom NUMBER := 0; BEGIN

-- Fetch employee row

SELECT *

INTO v_emp

FROM employees

WHERE employee_id = 5;

-- Calculate years of service
v_years_service := ROUND(MONTHS_BETWEEN(SYSDATE, v_emp.hire_date)/12, 1);

-- Classify service duration IF
v_years_service < 2 THEN
   v_service_status := 'Probationary'; ELSIF
v_years_service BETWEEN 2 AND 5 THEN
   v_service_status := 'Confirmed'; ELSE
   v_service_status := 'Senior Staff';
```

```
END IF;

-- Salary Analysis: CASE for pay grade
CASE
   WHEN v_emp.salary < 50000 THEN v_pay_grade := 1;
   WHEN v_emp.salary BETWEEN 50000 AND 69999 THEN v_pay_grade := 2;
   WHEN v_emp.salary BETWEEN 70000 AND 84999 THEN v_pay_grade := 3;
   WHEN v_emp.salary BETWEEN 85000 AND 99999 THEN v_pay_grade := 4;
   ELSE v_pay_grade := 5;
END CASE;

-- Raise check
IF v_years_service > 3 AND v_emp.salary < 85000 THEN
   v_raise_status := 'Raise recommended';
v_proposed_salary := v_emp.salary * 1.10; ELSE
   v_raise_status := 'No raise at this time';
v_proposed_salary := v_emp.salary;
END IF;

-- Department Summary: Loop through colleagues
FOR rec IN (
   SELECT first_name, last_name, salary
   FROM employees
   WHERE department = v_emp.department
     AND employee_id <> v_emp.employee_id
) LOOP
   v_total_colleagues := v_total_colleagues + 1;

   IF rec.salary > v_highest_salary THEN
v_highest_salary := rec.salary;
   END IF;

   IF rec.salary > v_emp.salary THEN
      v_higher_than_tom := v_higher_than_tom + 1;
   END IF;
END LOOP;

-- Final Report Output
DBMS_OUTPUT.PUT_LINE('========================================');
DBMS_OUTPUT.PUT_LINE('EMPLOYEE PROFILE REPORT');
DBMS_OUTPUT.PUT_LINE('AUCA — HR Department System');
DBMS_OUTPUT.PUT_LINE('========================================');
DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp.first_name || ' ' || v_emp.last_name ||
                 ' (ID: ' || v_emp.employee_id || ')');
DBMS_OUTPUT.PUT_LINE('Department: ' || v_emp.department);
DBMS_OUTPUT.PUT_LINE('Job Title: ' || v_emp.job_title);
DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(v_emp.hire_date, 'DD-MON-YY') ||
                 ' | Years of Service: ' || v_years_service);
```

```
DBMS_OUTPUT.PUT_LINE('Service Status: ' || v_service_status);
DBMS_OUTPUT.PUT_LINE('----------------------------------------');
DBMS_OUTPUT.PUT_LINE('Salary: RWF ' || v_emp.salary ||
                ' | Pay Grade: ' || v_pay_grade);
DBMS_OUTPUT.PUT_LINE('Raise Status: ' || v_raise_status);
DBMS_OUTPUT.PUT_LINE('Proposed Salary: RWF ' || v_proposed_salary);
DBMS_OUTPUT.PUT_LINE('----------------------------------------');
DBMS_OUTPUT.PUT_LINE('Dept. Colleagues: ' || v_total_colleagues);
DBMS_OUTPUT.PUT_LINE('Highest Salary in Dept: RWF ' || v_highest_salary);
DBMS_OUTPUT.PUT_LINE('Colleagues earning more than Tom: ' || v_higher_than_tom);
DBMS_OUTPUT.PUT_LINE('========================================');

END;

/
```